

Functional Programming Language

- the design of functional programming language is based on mathematical function.
 - the design of imperative language is based on von-neumann Architecture.
 - All Computations can be done by applying mathematical functions.
 - In functional languages, basic building blocks are functions.
 - the FPL are modeled on the concept of mathematical functions.
 - the programs are constructed by functions.
 - the purest functional programming languages use neither variables nor assignment statements.
 - the functional languages can be categorized into 2 types:
 1. Pure functional language - Ex: ML, Haskell
 2. Impure functional language - Ex: LISP
- Examples for functional languages:

- LISP
- SISCAL
- Haskell
- ML
- APL
- Scheme

Applications

- Machine Learning (ML)
- Natural Language Processing (NLP)
- Artificial Intelligence (AI)
- Knowledge representation
- used for building mathematical s/w.

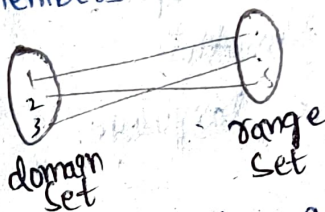
those that use
interpreter.
The scripting languages
run inside

Programming languages
are those that use
compiler.
These run inside

1) These languages are

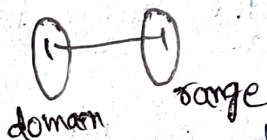
* Mathematical Function

→ It maps the members of domain set to members of range set.



→ The mapping is determined by a table or expression.

→ It maps an element of domain with same element available in range.



→ It is a process of mapping elements b/w domain & range.

Functions:

Square $(a) = a * a$ (for square function)

Cube $(x) = x * x * x$

$x(x) = x * x * x$ (for cube function)

Functional forms

→ There are two functional forms:

1. Functional Composition
2. Apply-to-all function

1. Functional Composition

→ It is a functional form.

→ It takes 2 functional parameters

→ Mathematically, ~~and~~ it is expressed as $h \equiv f \circ g$

Ex $CDR(X Y Z) \Rightarrow Y Z$ $(CDR(CAR(X Y Z))) \Rightarrow Y$

$CAR(X Y Z) \Rightarrow X$

~~$(CAR(CDR(AB))) X Y Z$~~

~~$CAR(CAR)$~~

2. Apply-to-all function

- It is one of the functional forms.
- It takes a single function parameter.
- The symbolic representation is λ

Ex: Form: a

for $h(x) \int x * x * x$

$a(h(2,3,4))$ yields (8,27,64)

* LISP

- LISP stands for List Processing.
- It is a programming language, used in AI.
- It is a functional programming language.
- It is high level programming language.
- The prefix notations are used to represent the expression.

Ex: $(+10\ 20) : 30$

$(-20\ 10) : 10$

- the code begins ~~but~~ with $>$

Ex: $> (+\ a\ b)$

- LISP was typeless language.
- There is no datatype declaration in LISP.
- there are only two type of data objects: - atom

- list.

- Atom: Symbol, identifier, numeric ~~data~~, Ex: a, 5

- list: lists are parenthesized collection of sublists or atoms

Ex: $(ab\ (cd)\ e)$
 $(10\ 20\ 30)$

CAR function: It returns first element

> (car '(1 2 3 4))

1

CDR fn: returns elements except first element

> (cdr '(1 2 3 4))

2 3 4

Cons fn: Combine two lists

> (cons 10 '(20 30))

(10 20 30)

length fn: > (length '(10 20 30 40 50)) = 5

Append: used to add the element in the list at the end

> (append '(10 20) '(40 50)) = (10 20 40 50)

Map:

> (map abs '(1 -2)) = (1 2)

> (map + '(1 2) (4 5)) = (5 7)

Imperative language

- 1) It is based on von-neumann architecture.
- 2) Efficiency is more.
- 3) It has complex Semantics.
- 4) Syntactic Structures are complex.
- 5) Concurrent execution is difficult to design.
- 6) Order of execution is very important.
- 7) Concurrent execution is difficult to use.
- 8) Concurrent program execution is difficult to understand.
- 9) Program is statically divided into concurrent parts by the programmer.
- 10) Ex: C, Java, Python, Ruby, JavaScript

Functional language

- 1) It is based on mathematical function.
- 2) Efficiency is less.
- 3) It has simple Semantics.
- 4) Syntactic structures are simple.
- 5) Concurrent execution is ~~easy~~ easy.
- 6) Order of execution is low in importance.
- 7) Concurrent execution is easy to use.
- 8) easy to understand.
- 9) Program is dynamically divided into concurrent parts by the programmer.
- 10) Ex: LISP, ML, Haskell

ML → Meta language	LISP	Scheme
→ It is static-scoped function PL	→ Same	→ Same
→ follows syntax	→ Same	→ Not follow syntax
→ Infix notation	→ Prefix	→ Prefix
→ Strongly typed language.	→ Typeless language.	→ typeless

Haskell

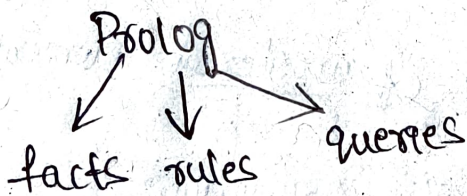
- Static scoped.
- follows syntax
- typed language

Prolog

- Prolog means 'logical programming language',
- It is a declarative language,
- In Prolog, we doesn't follow any procedure,
- In Prolog, ~~a~~ set of sentences are written in logical form,
- Logical Programming is a programming Paradigm
 - in which the set of sentences are written in logical form.
- The logic programs consists of facts & rules,

* Algorithm = Logic + Control

- Logic represents a logic program
 - in the form of facts & rules.
- Control represents how algorithm can be implemented by applying the rules



- * Facts: describe explicit relationship
- * Rules: defines implicit relationship.
- * Queries: conditions / questions.

Logical Operators

Name	Symbol	Example	Meaning
Negation	$\neg$	$\neg a$	a not b
Conjunction	$\cap$	$a \cap b$	a and b
Disjunction	$\cup$	$a \cup b$	a or b
Equivalence	$\equiv$	$a \equiv b$	a is equivalent to b
Implication	$\supset$	$a \supset b$	a implies b
	$\subset$	$a \subset b$	b implies a

Quantifiers

	Example	Meaning
--	---------	---------

Part III : Scripting Language

5.10 Pragmatics

Q.47 What are scripting languages ?

 [JNTU : Part A, Marks 2]

Ans. : A scripting language is a programming language designed for integrating and communicating with other programming languages. Some of the most widely used scripting languages are JavaScript, VBScript, PHP, Perl, Python, Ruby, ASP and Tcl.

Scripting languages, which can be embedded within HTML, commonly are used to add functionality to a Web page, such as different menu styles or graphic displays or to serve dynamic advertisements.

Sr. No.	Scripting Language	Programming Language
1.	Scripting languages are those that use interpreter.	Programming languages are those that use compiler.
2.	The scripting languages run inside another	These run independent of parent program or

	program. For example scripting languages execute within the web browser.	exterior.
3.	Designed to make coding fast and simple.	Designed to give full usage of a language.
4.	It does not create any .exe file.	It creates .exe file.
5.	Scripts are just piece of code.	Programming is making full code of program.
6.	These are easy to learn to write.	These are complex to learn.

Q.50 Enlist any five features of Python.

 **[JNTU : Part A, Marks 5]**

Ans. : Following are features of python :

- 1) Python is **free and open source** programming language. The software can be downloaded and used freely.
- 2) It **high level programming language**.
- 3) It is **simple and easy to learn**.
- 4) It is **portable**. That means python programs can be executed on various platforms without altering them.
- 5) It is an **object oriented programming language**.
- 6) It is **rich set of functionality** available in its huge standard library.
- 7) Python has a powerful set of **built-in data types and easy-to-use control constructs**.

Datatypes in Python

- Without datatype we can't store the data in variable.
- Python supports the dynamic datatypes
(At execution time datatype of variable will be decided)
- Python provides various datatypes. There are as follows:

1. None: It is a special type in python.

→ It indicates the object that will not contain any value.

→ In C, C++ we use null

Ex: x

2. Numeric

- int
- float
- complex
- bool

- Integer: Integer represents numbers without decimals

Ex: $x = 10$, or $y = -10$

To Verify: `type(x)` o/p: int

- Float: It represents numbers with decimals.

Ex: $x = 10.25$
 $y = -5.2$

- Complex: It is a combination of real part and imaginary part.

Ex: $x = 5 + 7j$

$$j = \sqrt{-1}$$

- boolean: It represents boolean values either True or False.

Ex: $x = \text{True}$ or $x = 1$

| $x = 8 > 2$
`print(x)` o/p: $x = \text{True}$ or 1

3. String: It is a collection of characters enclosed within single quote or double quote marks.
Ex: $x = \text{"Charan"}$

4. List:
 → It is like a array.
 → It is used to store more than one value in one variable.
 → used to store all elements (similar or dissimilar)
 → It is used to represent a group of elements in a single entity.

Ex: $x = [10, \text{"Cherry"}, 2.75]$

→ List is a mutable data type.
 - We can add elements → we can change
 - we can delete "
 - we can update "

5. Tuple: It is like list.

→ It is used to represent a group of elements in a single entity.

Ex: $x = (10, \text{"Cherry"}, 2.75)$

* Tuple is immutable

6. Set:

→ It is defined as a group of unordered elements.

Ex: $x = \{10, 20, 30, 40\}$
 $\text{print}(x)$ | o/p: $\{30, 10, 40, 20\}$

7. Dictionary or Mapping:

→ the elements are stored in key: ^{value} form.

Ex: $x = \{\text{"Rno"}: 1, \text{"Name"}: \text{"Cherry"}\}$

→ Instead of using loops, we use this.
 8. Range: To display some range of number.

Ex: $\text{range}(100)$

$\text{list}(\text{range}(100))$ - to convert range to list $[0, 1, 2, \dots, 99]$

$\text{list}(\text{range}(2, 100, 2))$ - To display even num $[2, 4, 6, 8, \dots, 98]$
 start end difference